test. That last point is important: even if you don't go as far as writing your tests before the code, it's well worth thinking about how you can test the code before you write it—what inputs to use, which conditions are likely to be problematic, how to stimulate the code in potentially problematic ways, and so on.

One of the best ways to design concurrent code for testing is to eliminate the concurrency. If you can break down the code into those parts that are responsible for the communication paths between threads and those parts that operate on the communicated data within a single thread, then you've greatly reduced the problem. Those parts of the application that operate on data that's being accessed by only that one thread can then be tested using the normal single-threaded techniques. The hard-to-test concurrent code that deals with communicating between threads and ensuring that only one thread at a time *is* accessing a particular block of data is now much smaller and the testing more tractable.

For example, if your application is designed as a multithreaded state machine, you could split it into several parts. The state logic for each thread, which ensures that the transitions and operations are correct for each possible set of input events, can be tested independently with single-threaded techniques, with the test harness providing the input events that would be coming from other threads. Then, the core state machine and message routing code that ensures that events are correctly delivered to the right thread in the right order can be tested independently, but with multiple concurrent threads and simple state logic designed specifically for the tests.

Alternatively, if you can divide your code into multiple blocks of *read shared data/ transform data/update shared data*, you can test the *transform data* portions using all the usual single-threaded techniques, because this is now just single-threaded code. The hard problem of testing a multithreaded transformation will be reduced to testing the reading and updating of the shared data, which is much simpler.

One thing to watch out for is that library calls can use internal variables to store state, which then becomes shared if multiple threads use the same set of library calls. This can be a problem because it's not immediately